

Cours de Programmation en C – EI-2I

Travaux Dirigés

TD3

Exercice 1 – Minimum

Écrire un algorithme qui renvoie l'indice du plus petit élément d'un tableau d'entiers.

Exercice 2 – Nombre d'occurrences

Écrire un programme qui saisit un tableau de N entiers (N fixé et défini dans le programme), ainsi qu'un entier e et qui renvoie le nombre de fois où l'entier e apparaît dans le tableau.

Exercice 3 – Palindrome

Écrire un algorithme qui prend un tableau d'entiers de taille N en entrée, et vérifie que ce tableau est un palindrome, c'est-à-dire qu'il se lit de la même façon dans un sens ou dans l'autre. Exemple : les tableaux 1,7,2,7,1 ou 3,4,4,3 sont des palindromes.

Exercice 4 – Remplacement des doublons

Soit un tableau T contenant des entiers strictement positifs. Écrire l'algorithme qui remplace tous les doublons par des 0. Exemple : le tableau 1,3,3,1,2,5,1 deviendra 1,3,0,0,2,5,0.

Exercice 5 – Multiplication de matrices

On considère deux matrices A et B de taille $m \times n$ et $p \times q$ avec m, n, p et q inférieurs à 10. On lit ces matrices dans un fichier qui contient (dans l'ordre)

- la taille de A (m et n)
- les valeurs de A , rangées par ligne
- la taille de B (p et q)
- les valeurs de B , rangées par ligne

1. Écrire un programme lisant les matrices A et B dans le fichier `matrices.txt`
2. Compléter le programme pour qu'il affiche la matrice B
3. Compléter le programme pour qu'il calcule et affiche le produit $C = A.B$.
Pour rappel, C existe si $n = p$ et est une matrice de taille $m \times q$ que l'on calcule par

$$C_{i,j} = \sum_{k=1}^n A_{i,k} B_{k,j}$$

4. Découper le programme en définissant et utilisant des fonctions

Remarque: les indices C commencent à 0, tandis que les indices des matrices commencent à 1.

Solution:

```
#include <stdio.h>
#include <stdlib.h>

#define TAILLE 10

int main(void)
{
    int i,j,k;           /* compteurs pour les boucles */
    int m,n,p,q;         /* tailles */
    float sum;           /* utilisé comme somme partielle pour le produit */
    float A[TAILLE][TAILLE];
    float B[TAILLE][TAILLE];
    float C[TAILLE][TAILLE];
    FILE* fichier;

    /* ouverture du fichier */
    fichier = fopen("matrices.txt", "r");
    if (fichier==NULL)
    {
        printf("Erreur_d'ouverture_de_fichier\n");
        exit(1);
    }

    /* lecture la matrice A */
    fscanf( fichier, "%d_%d", &m, &n);
    for( i=0; i<m; i++)
        for( j=0; j<n; j++)
            fscanf( fichier, "%f", &(A[i][j]) );

    /* lecture de la matrice B */
    fscanf( fichier, "%d_%d", &p, &q);
    for( i=0; i<p; i++)
        for( j=0; j<q; j++)
            fscanf( fichier, "%f", &(B[i][j]) );

    /* affichage de la matrice B */
    printf("matrice_B\n");
    for( i=0; i<p; i++)
    {
        for( j=0; j<q; j++)
        {
            printf("%f_", B[i][j]);
        }
        printf("\n");
    }

    /* test si C est calculable */
    if (n!=p)
    {
        printf("Les_dimensions_ne_correspondent_pas!\n");
        exit(1);
    }

    /* calcul de C */
    for( i=0; i<m; i++)
    {
        for( j=0; j<q; j++)
        {
            sum = 0;
            for( k=0; k<n; k++)
            {
                sum = sum + A[i][k]*B[k][j];
            }
            C[i][j] = sum;
        }
    }

    /* affichage de la matrice C */
    printf("\nMatrice_C\n");
    for( i=0; i<m; i++)
    {
        for( j=0; j<q; j++)
        {
            printf("%f_", C[i][j]);
        }
        printf("\n");
    }

    /* et on n'oublie pas la fermeture du fichier */
    fclose( fichier);

    return EXIT_SUCCESS;
}
```